Lecture 2: Data Representation I — Bits, Bytes & Integers

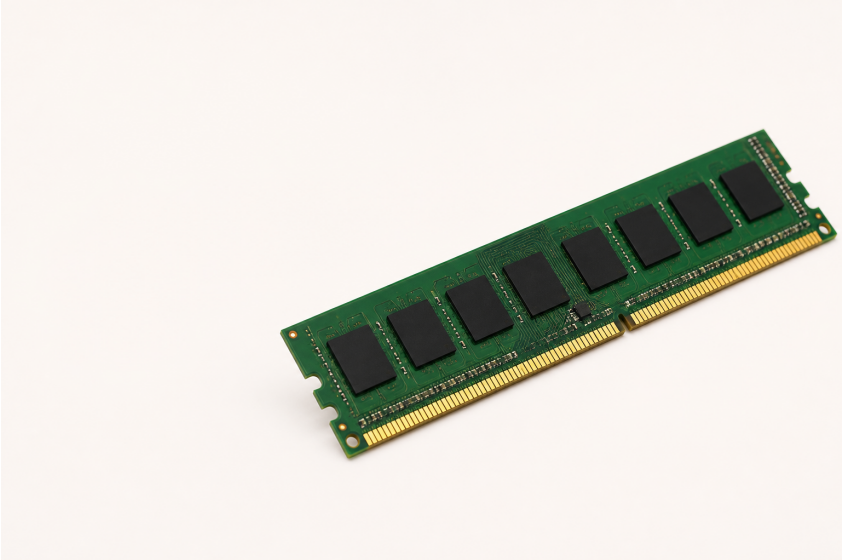
Session 2 · Mon Aug 31, 2026

Contents

From Physical Memory to One Byte

That is a **RAM module** — physical hardware, many chips, not one byte.

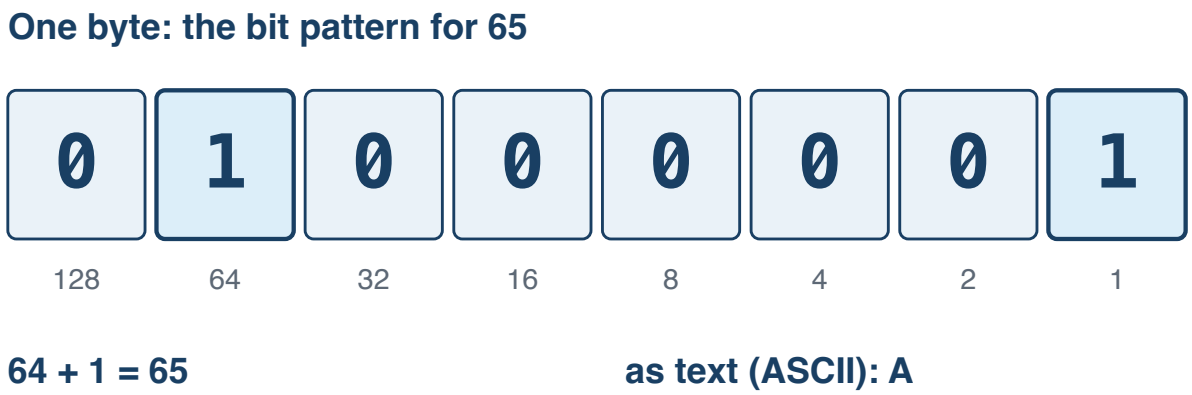
Memory is **byte-addressable**: every address names one **8-bit byte**, and a byte is the smallest unit a C program can address on its own.



One byte = 8 bits. Today is about what fits in one, and how to work on it.

One Byte, Two Readings

Start with one byte: eight switches that are each either 0 or 1.



The computer stores only the bits; the program decides how to read them.

Characters Are Just Numbers

The rule that turns 65 into `A` is **ASCII**: a table agreed in the 1960s giving every character a number from 0 to 127.

Characters	Codes	Worth knowing
`'0'`–`'9'`	48–57 (`0x30`–`0x39`)	consecutive, so `c - '0'` gives the digit
`'A'`–`'Z'`	65–90 (`0x41`–`0x5A`)	consecutive
`'a'`–`'z'`	97–122 (`0x61`–`0x7A`)	exactly 32 more than the capitals

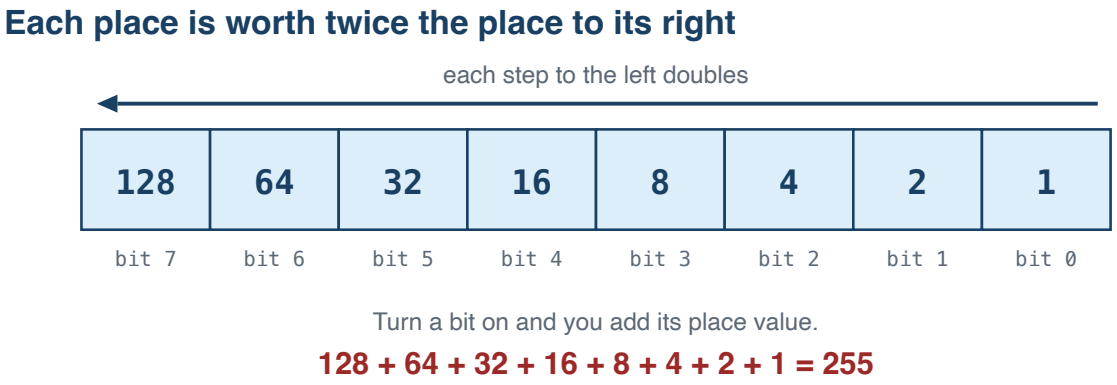
```c
char c = '7';
int  d = c - '0';                    // 7
```

```c
int  is_digit = (c >= '0' && c <= '9');  // 1
```

In C the literal `'A'` **is** the number 65. Nothing clever is happening.

Counting in Binary

Why does `01000001` mean 65? Because each place is worth twice the one to its right — the same idea as decimal's ones, tens and hundreds, with 2 instead of 10.



Counting Up

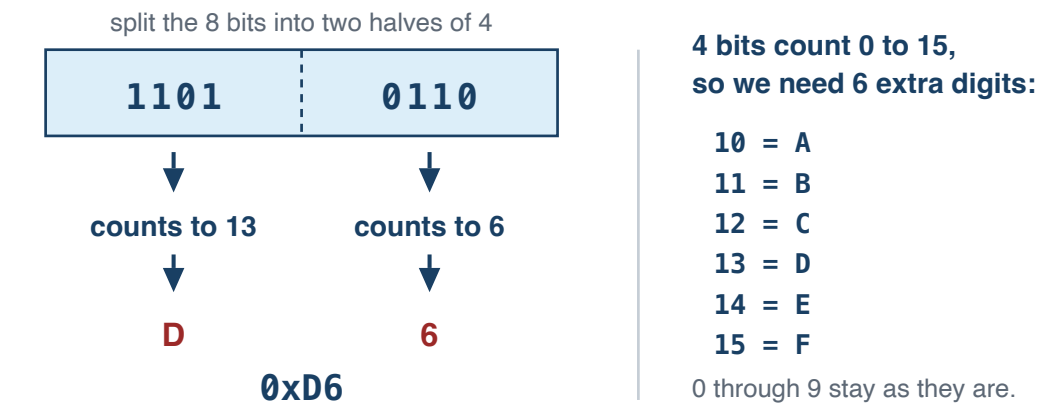
Counting works exactly the way it does in decimal — when a column runs out of digits, it rolls over and carries into the next one:

```
0 = 0000    3 = 0011    6 = 0110    9  = 1001
1 = 0001    4 = 0100    7 = 0111    10 = 1010
2 = 0010    5 = 0101    8 = 1000    11 = 1011
```

Each extra bit doubles the patterns available: 8 bits give **2^8 = 256**. So an unsigned byte runs 0–255, and an N-bit unsigned value runs 0 to `2^N - 1` — its **UMax**, modulo `2^N`.

Writing Bit Patterns: Hex

Eight ones and zeros is easy to miscount. Four bits count 0–15 — one **hexadecimal** digit — so a byte is two digits you can still read as bits.



`0x` just means "hex follows". `0x2A` is 42; `0xFF` is 255.

> **Note**
>
> `0b1100` is slide notation. GCC accepts it, but binary literals are C23; in ISO C17 write `0xC`.

The Same Ideas in Python

C

```c
int x = 65;
printf("%d\n", x);      // 65
printf("%c\n", x);      // A

unsigned a = 12, b = 10;
printf("%u\n", a & b);  // 8
printf("%u\n", a << 2); // 48
```

Python

```python
x = 65
print(x)            # 65
print(chr(x))       # A

a, b = 12, 10
print(a & b)        # 8
print(a << 2)       # 48
```

The operators are the same. The difference is the box: a C value has a fixed width, and a Python integer just grows.

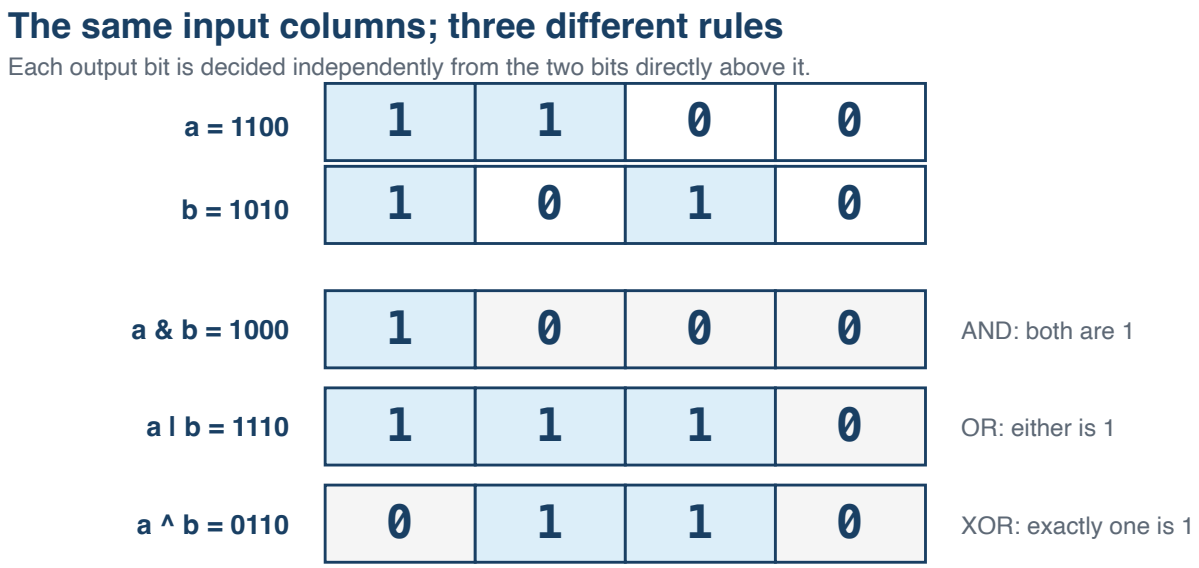
Bit-Level Operations

For these bit patterns, C provides six bitwise operators:

Op	Name	Example
`&`	AND	`0b1100 & 0b1010 = 0b1000`
`|`	OR	`0b1100 | 0b1010 = 0b1110`
`^`	XOR	`0b1100 ^ 0b1010 = 0b0110`
`~`	NOT	`~0b1100 = 0b1111...11110011` (every bit flips, including the leading zeros)
`<<`	Left shift	`0b0011 << 2 = 0b1100`
`>>`	Right shift	`0b1100 >> 2 = 0b0011`

> **Warning**
>
> Right shift of signed integers is **implementation-defined** in C. GCC uses **arithmetic** right shift (preserves sign bit). Don't rely on this unless you're sure of your compiler.

Bitwise Operations, One Column at a Time



De Morgan's Laws

Two rules let you rewrite any mix of `&`, `|` and `~`:

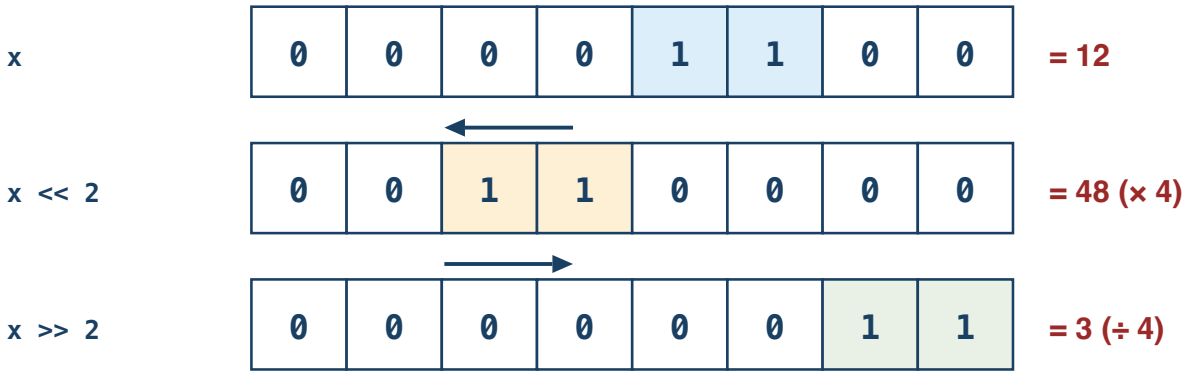
```
~(x & y)  ==  ~x | ~y          ~(x | y)  ==  ~x & ~y
```

x	y	x & y	~(x & y)	~x | ~y
0	0	0	1	1
0	1	0	1	1
1	0	0	1	1
1	1	1	0	0

The last two columns agree on every row, so the two expressions are the same.

Shifting Bits

Shifting slides every bit sideways

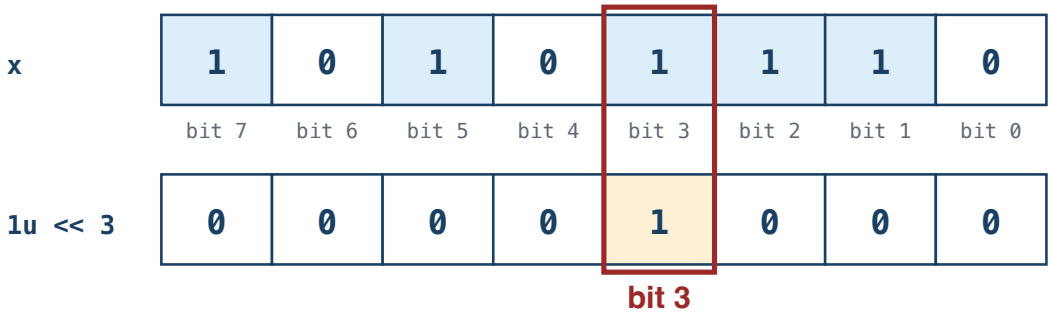


Bits pushed off the end are gone for good; zeros come in behind them.

Shifting left by n multiplies by 2^n. Shifting an unsigned value right by n divides by 2^n.

Test, Set, Clear, Toggle

1u << 3 builds a mask with exactly one bit set



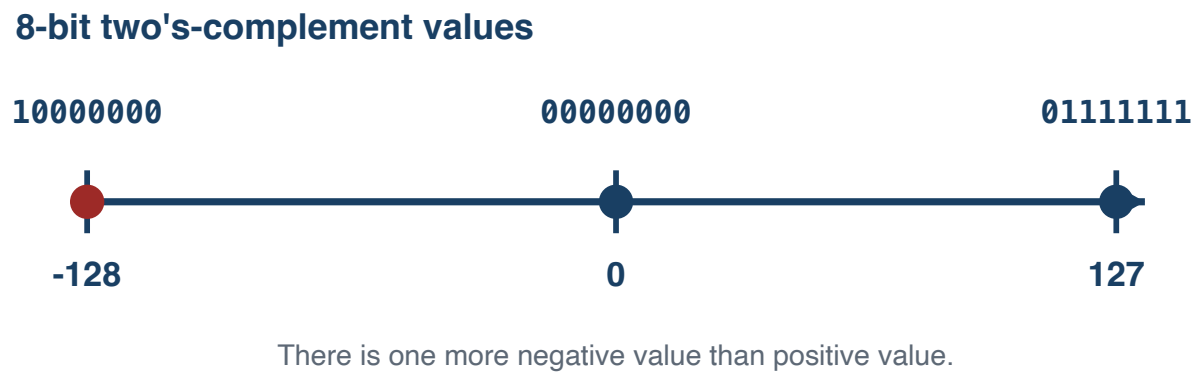
Bit 0 is the rightmost — the ones place. The mask is 0 everywhere except the bit you want.

```
unsigned x = 0xAE;              // 10101110
x |=  (1u << 3);                // SET     — bit 3 becomes 1
x &= ~(1u << 3);                // CLEAR   — bit 3 becomes 0
x ^=  (1u << 3);                // TOGGLE  — bit 3 flips
if (x & (1u << 3)) { }          // TEST    — nonzero when bit 3 is 1
```

Those four are most of Lab 1. Keep the value **unsigned** — shifting into a sign bit is undefined.

Two's Complement on Our Platform

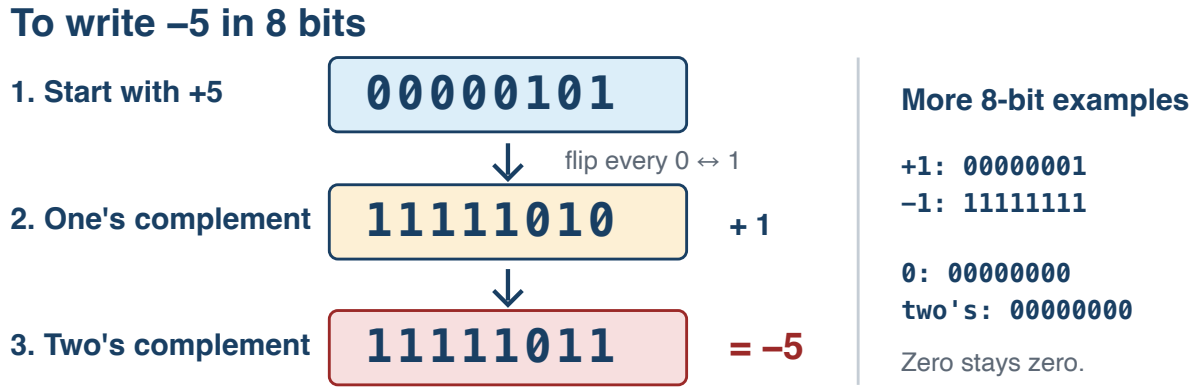
One's complement flips every bit. **Two's complement** writes `-x` by flipping the bits of `x` and adding 1, and covers `-2^(N-1)` to `2^(N-1) - 1`:



`TMin = -TMax - 1` — there is no positive 128 in 8 bits, so negating `TMin` overflows and is undefined.

How We Get There: +5 Becomes -5

Always name the width — every pattern here is 8 bits. Flip every bit, add 1.

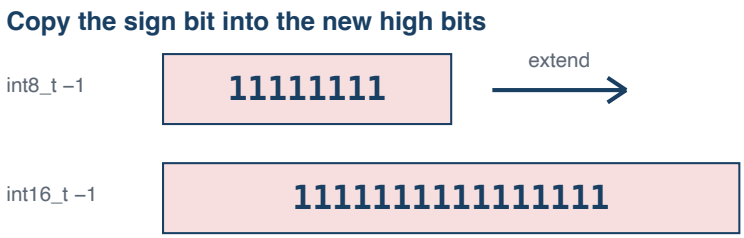


A ninth bit is discarded: 8 bits keep only their rightmost 8.

Sign Extension

When converting a smaller signed type to a larger one, copy the sign bit:

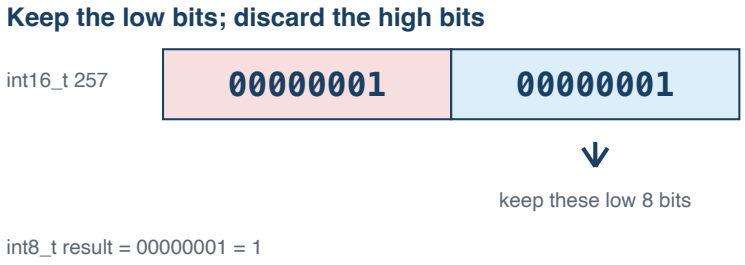
```
int8_t  x = -1;   // 0b11111111
int16_t y = x;    // 0b1111111111111111 = -1 ✓
```



Truncation

When converting larger to smaller, drop the high bits:

```
int16_t  x = 257;
uint8_t  y = x;    // 1: conversion to unsigned is modulo 256
```



Size of Integer Types

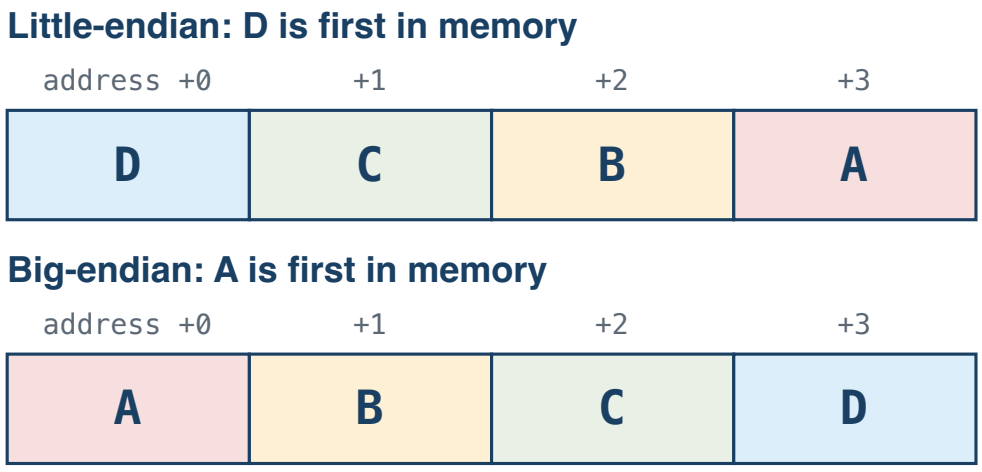
```
printf("char:      %zu\n", sizeof(char));      // 1 C byte (not always 8 bits)
printf("short:     %zu\n", sizeof(short));     // 2
printf("int:       %zu\n", sizeof(int));       // 4
printf("long:      %zu\n", sizeof(long));      // 8 (LP64)
printf("long long: %zu\n", sizeof(long long)); // 8
printf("pointer:   %zu\n", sizeof(void*));     // 8
```

Tip

Use `sizeof` and fixed-width types (`int32_t`, `uint64_t`) when exact sizes matter. Never assume `int` is 4 bytes; use `CHAR_BIT` when bit width matters. Plain `char` is its own trap: whether it is signed is implementation-defined, so say `signed char` or `unsigned char` when it matters.

Byte Ordering (Endianness)

An `int` occupies 4 consecutive bytes, so those 4 bytes have to be laid out in some order. **Endianness** is that order. One question matters: **which byte goes at the lowest address?**




```
int x = 1;   // ask the machine: which byte of x sits at the lowest address?
if (*(unsigned char *)&x == 1) printf("Little-endian\n");
else                           printf("Big-endian\n");
```

AI Systems Connection

Quantization is one important reason large models can fit on more hardware. A common symmetric int8 scheme represents a block of weights with 8-bit signed values and a scale factor:

```
real_value  ≈  scale × int8_value          scale = max_abs / 127
```

Everything today shows up here. Two int8 values need 16 bits for their product, so a long dot product accumulates into something wider — often `int32_t`. And if a signed weight is loaded as *unsigned* into a wider register, sign extension does not happen, negatives become huge positives, and the answer is quietly wrong.

Try it: quantize a float array with `scale = max_abs / 127`, dequantize, and measure the error.

Practice

1. Write `0xD6` in binary, and `10110010` in hex, without using a computer.
2. What does `x & 1` tell you about `x`? And what does `x & (x - 1)` do? Try both on `0b0110` by hand.
3. Write one expression that clears bit 5 of `unsigned x`, and one that tests whether bit 0 and bit 7 are both set.
4. What is `TMax` for a 32-bit `int`? What is `TMin`?
5. Write a C expression that checks if your machine is little-endian.
6. Is left-shifting a 32-bit `int` by 32 positions defined in C? Why not?
7. Why is `-TMin` not representable in the same signed type?

Reading

- CS:APP §2.1 (Information Storage)
- CS:APP §2.2 (Integer Representations)